CAPACITACIÓN QA AUTOMATION · PARTE 2 · SESIÓN 2 DE 4

De Page Object a Screenplay

El escenario habla de personas y objetivos, no de botones. Reescribimos el login con un actor que tiene habilidades, realiza tareas y hace preguntas.

Instructor **Giancarlo Palomino**

Miércoles 28 de octubre · 17:30 a 18:45

Playwright 1.64 · Cucumber 13 · TypeScript

75 MINUTOS

Hoy cambiamos la forma de pensar las pruebas

10 min — **Revisión de los PR de ayer**
Lo que más se repitió en los ejercicios 1 y 2.

10 min — **Por qué Screenplay**
El escenario habla de personas y objetivos, no de botones.

25 min — **En vivo: Actor, Ability, Interaction, Task y Question**
El login del proyecto base, reescrito en Screenplay.

30 min — **Ejercicio 3 · ¿El ordenamiento ordena?**
Con una `Question` reutilizable. Entrega: PR `feature/s2-<tu-nombre>`.

REQUISITO

Tu rama de ayer con `npm test` en verde. Si no llegaste, parte de `develop`: el ejercicio de hoy no depende de los de ayer.

AL TERMINAR SABRÁS

Separar **qué** quiere lograr el usuario (tareas) de **cómo** se hace en la pantalla (interacciones) y de **qué** ve (preguntas).

LO QUE VIMOS EN LOS PULL REQUEST

Cuatro observaciones que valen para toda la semana

`strict mode violation · resolved to 3 elements`

Un selector que encuentra de más

En cada producto, la imagen y el título también tienen rol *button*. Hay que nombrarlo: `getByRole('button', { name: 'Add to cart' })`.

`let productos: string[] = []`

Estado fuera del World

Una variable global se comparte entre escenarios. Lo que recuerda un escenario va en `this` (el World).

`await page.waitForTimeout(2000)`

Espera fija "para que pase"

Si hay que esperar, falta una aserción web-first: `toHaveText`, `toHaveCount`, `not.toBeVisible`.

`page.locator(...) dentro del step`

Selectores en los steps

Funciona hoy, se rompe mañana en tres archivos. Es justo el problema que resuelve el patrón de hoy.

01

Por qué Screenplay

El Page Object organiza la pantalla. Screenplay organiza lo que hace el usuario.

PAGE OBJECT A ESCALA

Cuando la clase de una pantalla sabe demasiado

PAGE OBJECT · ORGANIZADO POR PANTALLA

proyecto base · src/pages/LoginPage.ts — ANTES

```ts
export class LoginPage {
  constructor(private readonly page: Page) {}

  async abrir() { await this.page.goto('/'); }

  async ingresar(usuario: string, clave: string) {
    await this.page.locator('[data-test="username"]').fill(usuario);
    await this.page.locator('[data-test="password"]').fill(clave);
    await this.page.locator('[data-test="login-button"]').click();
  }

  mensajeError() { return this.page.locator('[data-test="error"]'); }
}
```

Selectores, acciones y consultas en la misma clase. Una compra cruza cuatro pantallas: ¿en cuál vive?

SCREENPLAY · ORGANIZADO POR OBJETIVO

src/steps/login.steps.ts — DESPUÉS

```ts
Given('que inicié sesión como {string}',
  async function (this: CustomWorld, usuario: string) {
    await this.actor.intenta(IniciarSesion.como(usuario));
  });

Then('veo el catálogo de productos',
  async function (this: CustomWorld) {
    expect(await this.actor.pregunta(TextoDe.el(Pantalla.TITULO)))
      .toBe('Products');
  });
```

El step se lee como el escenario: *el actor intenta iniciar sesión; el actor pregunta el título.*

LA IDEA EN UNA FRASE

Un **actor** con **habilidades** **intenta** tareas y **pregunta** lo que ve. Las tareas no saben de selectores; los selectores no saben de tareas.

EL VIAJE DE UNA FRASE, ANTES Y AHORA

Page Object frente a Screenplay

PAGE OBJECT (SESIÓN 1)

FEATURE
Gherkin
"ingreso con el usuario..."

→

STEP
When(...)
crea el Page Object

→

PAGE OBJECT
LoginPage
selectores + acciones + consultas, todo junto

→

PLAYWRIGHT
page
el navegador

SCREENPLAY (DESDE HOY)

FEATURE
Gherkin
sin cambios

→

STEP
this.actor
solo delega

→

TASK
IniciarSesion
el objetivo de negocio

→

INTERACTION
Escribir · Clic
una acción + un Target

→

ABILITY
NavegarLaWeb
la Page de Playwright

SI CAMBIA EL ESCENARIO
Tocas el step.

SI CAMBIA LA PANTALLA
Tocas un Target en `ui/` : una línea.

SI CAMBIA EL FLUJO
Tocas una tarea, y todos los escenarios que la usan se enteran.

EL VOCABULARIO DE SCREENPLAY

Cinco piezas, cada una con un solo trabajo

QUIÉN

Actor

La persona que usa el sistema. Tiene habilidades, intenta cosas y hace preguntas.

```
Actor.llamado('Comprador')
```

CON QUÉ

Ability

Lo que le permite actuar. Aquí: usar un navegador. Mañana: llamar a una API.

```
NavegarLaWeb.con(page)
```

CÓMO

Interaction

Una sola acción sobre la pantalla, sobre un Target.

`Clic.en(…)` · `Escribir.elValor(…)`

PARA QUÉ

Task

Un objetivo de negocio. Compone interacciones u otras tareas.

`IniciarSesion.como(…)`

QUÉ VE

Question

Lee un valor de la pantalla. Nunca cambia nada.

`TextoDe.el(…)`

TARGET · EL QUE FALTABA

Un elemento con nombre legible y su selector: `Target.el('campo usuario').ubicadoPor('[data-test="username"]')`. Los selectores viven solo en `src/screenplay/ui/`.

SE LEE COMO UNA FRASE

```
actor.intenta(IniciarSesion.como('standard_user'))
actor.pregunta(TextoDe.el(Pantalla.TITULO))
```

SRC/SCREENPLAY/CORE/ACTOR.TS

El actor: habilidades, tareas y preguntas

src/screenplay/core/Actor.ts · extracto — TYPESCRIPT

```ts
export class Actor {
  private readonly habilidades = new Map<object, Ability>();

  static llamado(nombre: string): Actor {
    return new Actor(nombre);
  }

  /** Le da al actor una o más habilidades. */
  puede(...habilidades: Ability[]): this {
    for (const habilidad of habilidades) {
      this.habilidades.set(habilidad.constructor, habilidad);
    }
    return this;
  }

  /** Realiza, en orden, las tareas o interacciones indicadas. */
  async intenta(...actividades: Performable[]): Promise<void> {
    for (const actividad of actividades) await actividad.realizadaPor(this);
  }

  /** Hace una pregunta y devuelve la respuesta. */
  pregunta<T>(pregunta: Question<T>): Promise<T> {
    return pregunta.respondidaPor(this);
  }
}
```

core/Performable.ts · core/Question.ts — CONTRATOS

```ts
export interface Performable {
  realizadaPor(actor: Actor): Promise<void>;
}

export interface Question<T> {
  respondidaPor(actor: Actor): Promise<T>;
}
```

PERFORMABLE

Lo implementan las **Interactions** y las **Tasks**: para el actor son lo mismo, "algo que puedo realizar".

QUESTION<T>

Devuelve un valor tipado: `string`, `number`, `string[]`… y el step decide qué esperar.

EN INGLÉS

`intenta` = `attemptsTo` y `pregunta` = `answer` en Serenity/JS.

LA ÚNICA PUERTA A PLAYWRIGHT

La habilidad envuelve la página; el Target, el selector

src/screenplay/habilidades/NavegarLaWeb.ts — ABILITY

```ts
export class NavegarLaWeb extends Ability {
  private constructor(readonly page: Page) {
    super();
  }

  static con(page: Page): NavegarLaWeb {
    return new NavegarLaWeb(page);
  }

  /** Atajo para interacciones y preguntas. */
  static de(actor: Actor): NavegarLaWeb {
    return actor.usar(NavegarLaWeb);
  }

  localizar(target: Target): Locator {
    return target.localizarEn(this.page);
  }
}
```

src/screenplay/ui/PantallaLogin.ts — TARGETS

```ts
export const PantallaLogin = {
  USUARIO: Target.el('campo usuario')
    .ubicadoPor('[data-test="username"]'),
  CLAVE: Target.el('campo clave')
    .ubicadoPor('[data-test="password"]'),
  INGRESAR: Target.el('botón ingresar')
    .ubicadoPor('[data-test="login-button"]'),
};
```

src/screenplay/ui/Catalogo.ts · extracto — TARGET DINÁMICO

```ts
agregar: (nombre: string) =>
  Target.el(`botón agregar de ${nombre}`).ubicadoCon((page) =>
    page.locator(producto, { hasText: nombre })
      .getByRole('button', { name: 'Add to cart' })),
```

DÓNDE VIVE CADA COSA

Selectores: solo en `ui/`. La `Page`: solo dentro de la habilidad.

UNA ACCIÓN, UN TARGET

Interacciones: los verbos más pequeños

src/screenplay/interacciones/Clic.ts — INTERACTION

```ts
export class Clic implements Performable {
  private constructor(private readonly target: Target) {}

  static en(target: Target): Clic {
    return new Clic(target);
  }

  async realizadaPor(actor: Actor): Promise<void> {
    await NavegarLaWeb.de(actor).localizar(this.target).click();
  }
}
```

src/screenplay/interacciones/Escribir.ts — INTERACTION

```ts
export class Escribir implements Performable {
  private constructor(
    private readonly valor: string,
    private readonly target: Target,
  ) {}

  static elValor(valor: string) {
    return { en: (target: Target) => new Escribir(valor, target) };
  }

  async realizadaPor(actor: Actor): Promise<void> {
    await NavegarLaWeb.de(actor).localizar(this.target).fill(this.valor);
  }
}
```

CONSTRUCTOR PRIVADO

Se crean con un método estático que se lee como frase: `Escribir.elValor('Ana').en(Checkout.NOMBRE)`.

LAS DEL PROYECTO

`Navegar`, `Clic`, `Escribir`, `Seleccionar`, `VolverAtras`.

ESPERAS INCLUIDAS

`click()` y `fill()` siguen esperando solos a que el elemento esté listo.

UN OBJETIVO DE NEGOCIO

Las tareas componen interacciones… y otras tareas

src/screenplay/tareas/IngresarCredenciales.ts — TASK

```ts
export class IngresarCredenciales implements Performable {
  private constructor(
    private readonly usuario: string,
    private readonly clave: string,
  ) {}

  static de(usuario: string, clave: string) {
    return new IngresarCredenciales(usuario, clave);
  }

  async realizadaPor(actor: Actor): Promise<void> {
    await actor.intenta(
      Escribir.elValor(this.usuario).en(PantallaLogin.USUARIO),
      Escribir.elValor(this.clave).en(PantallaLogin.CLAVE),
      Clic.en(PantallaLogin.INGRESAR),
    );
  }
}
```

src/screenplay/tareas/IniciarSesion.ts · extracto — TASK

```ts
export const CLAVE_DEMO = 'secret_sauce';

static como(usuario: string, clave = CLAVE_DEMO) {
  return new IniciarSesion(usuario, clave);
}

async realizadaPor(actor: Actor): Promise<void> {
  await actor.intenta(
    Navegar.a('/'),
    IngresarCredenciales.de(this.usuario, this.clave),
  );
}
```

NOMBRE = INTENCIÓN

`IniciarSesion`, no `LlenarFormularioYClic`. Si el login pasa a ser con huella, cambia el interior, no el nombre.

PREGUNTAR SIN TOCAR

La pregunta lee; el step decide qué esperar

src/screenplay/preguntas/TextoDe.ts — QUESTION

```ts
export class TextoDe implements Question<string> {
  private constructor(private readonly target: Target) {}

  static el(target: Target): TextoDe {
    return new TextoDe(target);
  }

  async respondidaPor(actor: Actor): Promise<string> {
    return (await NavegarLaWeb.de(actor)
      .localizar(this.target).innerText()).trim();
  }
}
```

ESPERA

`innerText()` espera a que el elemento exista. Para listas que cambian, en la sesión 3: `expect.poll`.

src/steps/login.steps.ts · el login reescrito — STEPS

```ts
Given('que estoy en la página de login', async function (this: CustomWorld) {
  await this.actor.intenta(Navegar.a('/'));
});

When('ingreso con el usuario {string} y la clave {string}',
  async function (this: CustomWorld, usuario: string, clave: string) {
    await this.actor.intenta(IngresarCredenciales.de(usuario, clave));
  });

Then('veo el catálogo de productos', async function (this: CustomWorld) {
  expect(await this.actor.pregunta(TextoDe.el(Pantalla.TITULO)))
    .toBe('Products');
});

Then('veo el mensaje de error {string}',
  async function (this: CustomWorld, mensaje: string) {
    expect(await this.actor.pregunta(TextoDe.el(Pantalla.ERROR)))
      .toContain(mensaje);
  });
```

25 MINUTOS, SIN COPIAR Y PEGAR

Guion de la demo: del Page Object al actor

1	0–3	Correr `npm test`: 3 escenarios en verde. Esa es la red de seguridad del cambio.
2	3–8	`core/`: `Ability`, `Performable`, `Question`, `Target` y el `Actor` con `puede`, `intenta` y `pregunta`.
3	8–11	`NavegarLaWeb` y los Targets de `PantallaLogin` y `Pantalla`.
4	11–15	`Navegar`, `Escribir` y `Clic`.
5	15–19	`IngresarCredenciales` e `IniciarSesion`; `TextoDe`.
6	19–23	World con `actor`; en `Before`: `Actor.llamado('Comprador').puede(NavegarLaWeb.con(this.page))`. Reescribir los 4 steps.
7	23–25	`login.steps.ts` ya no importa `LoginPage`. `npm run typecheck` y `npm test`: los mismos 3 en verde.

```
$ npm run typecheck
$ npm test
...
3 scenarios (3 passed)
15 steps (15 passed)
```

LO IMPORTANTE

El `login.feature` **no cambia ni una letra**. Cambió todo lo de abajo y el escenario sigue diciendo lo mismo.

SI ALGO FALLA EN VIVO

Mejor: leer el rojo juntos. Lo más probable es un `await` que falta dentro de `intenta`.

DÓNDE VA CADA COSA

Una carpeta por pieza, una regla por carpeta

steps/	Traducen Gherkin. Solo `this.actor.intenta` y `this.actor.pregunta`.
tareas/	Objetivos de negocio. Sin selectores, sin `page`.
preguntas/	Leen la pantalla y devuelven valores. No cambian nada.
interacciones/	Una acción de Playwright sobre un Target.
ui/	Targets: los únicos selectores del proyecto.
habilidades/	NavegarLaWeb: la única que conoce la `Page`.
core/	Actor, Ability, Performable, Question, Target. No cambia nunca.

```
src/
├── screenplay/
│   ├── core/          Actor.ts  Ability.ts  Performable.ts
│   │                  Question.ts  Target.ts
│   ├── habilidades/   NavegarLaWeb.ts
│   ├── interacciones/ Navegar.ts  Clic.ts  Escribir.ts
│   ├── tareas/        IniciarSesion.ts  IngresarCredenciales.ts
│   ├── preguntas/     TextoDe.ts
│   └── ui/            PantallaLogin.ts  Pantalla.ts
├── steps/             login.steps.ts
└── support/           world.ts  hooks.ts
```

NOMBRES EN ESPAÑOL

Tareas como verbos en infinitivo (`IniciarSesion`), preguntas como lo que devuelven (`TextoDe` , `ContadorDelCarrito`).

3
NIVEL

VERIFICAR DE VERDAD · CON SCREENPLAY

¿El ordenamiento ordena?

30 min
Dificultad

OBJETIVO

El actor ordena el catálogo y una **Question reutilizable** lee lo que se ve y comprueba el orden. Sirve para los cuatro criterios, y el jueves encontrará un bug.

CRITERIOS DE ACEPTACIÓN

1. Un *Esquema* cubre `az`, `za`, `lohi` y `hilo`.
2. Ordenar es una Task del actor; los steps no usan `page` ni selectores.
3. Una Question lee nombres o precios (`"$7.99"` → `7.99`) y los compara con una copia ordenada por el código.
4. Ningún valor esperado escrito a mano. Si falla, se ven las dos listas.

Aprendes: una Question con lógica, `Seleccionar` como interacción, `allInnerTexts()` y aserciones sobre colecciones.

features/ordenamiento.feature — GHERKIN

```gherkin
@ejercicio3
Esquema del escenario: Ordenar el catálogo por <criterio>
  Dado que inicié sesión como "standard_user"
  Cuando ordeno los productos por "<criterio>"
  Entonces los productos quedan ordenados por "<criterio>"
  Ejemplos:
    | criterio |
    | az       |
    | za       |
    | lohi     |
    | hilo     |
```

PISTAS

Orden: `[data-test="product-sort-container"]` · nombres: `inventory-item-name` · precios: `inventory-item-price`. La Question puede devolver `{ obtenido, esperado }` y el step solo compara.

LEER EL ROJO

Seis tropiezos al pasar a Screenplay

```
el paso pasa y no hizo nada
```

Falta un await

`this.actor.intenta(...)` sin `await`: el step termina antes que la tarea. Lo mismo dentro de `realizadaPor`.

```
Comprador no tiene la habilidad NavegarLaWeb
```

Actor sin habilidad

El `Before` no hizo `.puede(NavegarLaWeb.con(this.page))`, o se creó otro actor en el step.

```
Cannot read properties of undefined (reading 'intenta')
```

this se perdió

Step escrito con función flecha: no tiene el World. Usa `async function (this: CustomWorld)`.

`page.locator en una Task`

Tarea con selectores

Funciona, pero rompe el patrón. El selector va a un Target en `ui/`.

`Question que hace clic`

Pregunta que cambia cosas

Una pregunta solo lee. Si hay que hacer algo antes, es una tarea en el *Cuando*.

`expect dentro de una Task`

Verificar en la tarea

Las tareas hacen; los *Entonces* verifican con preguntas.

PARA LLEVARTE · ENTREGA: PR FEATURE/S2-<TU-NOMBRE>

Un actor que intenta tareas y hace preguntas

1 · Qué, cómo y qué veo
Tareas para objetivos, interacciones para acciones, preguntas para leer.

2 · Un lugar por cosa
Selectores en `ui/`, la Page en la habilidad, el flujo en las tareas.

3 · Mañana
El actor en el World, una compra completa, tablas de datos y evidencia cuando algo falla.

Tarea: terminar el **ejercicio 3** con `npm test` en verde

Enunciados: **docs/06-ejercicios.md**